

Trabajo Práctico 4

TEMAS

- Creación de excepciones.

OBJETIVOS

Que el estudiante adquiera las siguientes competencias específicas:

- Desarrollar excepciones propias para utilizar en la construcción de programas.
- Usar, en forma eficiente, la diagramación UML en la documentación de aplicaciones de software.

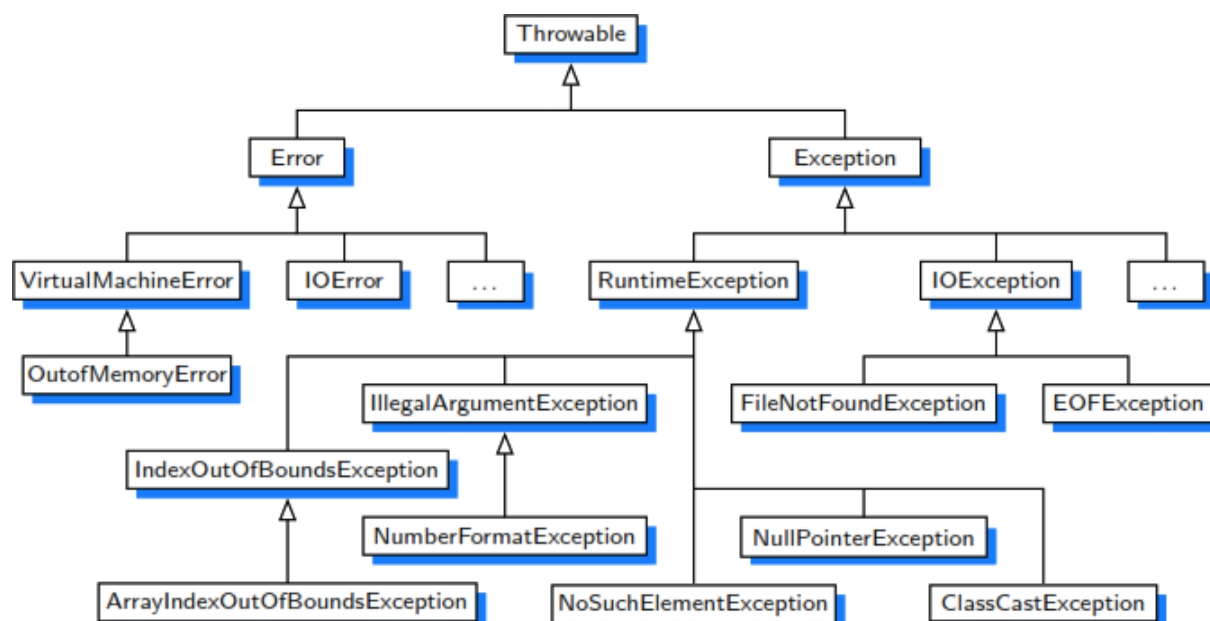
RECURSOS BIBLIOGRÁFICOS

Libro	
GOODRICH – TAMASSIA.	Estructuras de Datos y Algoritmos en Java. Editorial Continental. Mexico. 2002
DEITEL P.J.	JAVA- como programar"- Pearson- Prentice Hall. Septima edición 2008
WEISS MARK ALLEN.	Estructuras de datos en JAVA. Pearson- 2013
Software	Plataforma Java y última versión del IDE NetBeans

CONSIGNAS

- Identifique clases, atributos, métodos y objetos para resolver el problema aplicando POO. Confeccione una tabla con lo identificado.
- Aplique diagramación de clases UML.
- Expresar el algoritmo solución en diagramas de flujo, para cada uno de los métodos identificados.
- Desarrolle la codificación en Java de las clases, utilizando el entorno de desarrollo (IDE) Netbeans.
- Envío de la codificación mediante plataforma CUV .
- La resolución de la identificación de componentes del problema, la diagramación de flujo y la diagramación UML, deben estar completadas en la carpeta del alumno.

PARTE DE LA JERARQUÍA DE CLASES DEL TIPO THROWABLE



Trabajo Práctico 4

EJERCICIOS

Ejercicio 1.

Una empresa desea llevar un control sobre los sueldos de sus empleados.

Se debe implementar una clase Empleado con los siguientes atributos: nombre (String), dni (int) y sueldo (double).

Se requiere:

- Que el sueldo no sea menor al salario mínimo (\$120.000). En caso contrario, lanzar una excepción SueldoBajoException.
- Que el DNI sea un número positivo de 7 u 8 cifras. En caso contrario, lanzar una excepción DniInvalidoException.

Ambas excepciones deben heredar de una excepción base EmpleadoException.

Ejercicio 2.

Crear una clase Banco que contenga un método retirar(double monto) y un atributo saldo.

- Si el monto a retirar es mayor que el saldo, lanzar una excepción SaldoInsuficienteException.
- Si el monto es negativo, lanzar una excepción MontoInvalidoException.

Ambas excepciones deben derivar de una clase OperacionBancariaException.

Desde el main, se debe crear una cuenta con saldo inicial y simular varios retiros, usando múltiples bloques catch para capturar las excepciones.

Ejercicio 3.

Se desea implementar una clase Vehiculo que tenga los atributos: patente (String) y añoFabricacion (int).

- Si el año de fabricación es mayor al actual, lanzar una excepción AnioFuturoException.
- Si el año de fabricación es menor al año 1950, lanzar una excepción AnioMuyAntiguoException.

Ambas excepciones deben heredar de una clase VehiculoException.

El programa debe pedir los datos por teclado e instanciar un objeto Vehiculo, tratando las excepciones adecuadamente.

Ejercicio 4.

Diseñar una aplicación para una biblioteca. Crear una clase Libro con los atributos: titulo (String), autor (String), anioPublicacion (int).

- Si el año de publicación es posterior al actual, lanzar FechaInvalidaException.
- Si el título o autor están vacíos o nulos, lanzar DatoVacioException.

Ambas deben derivar de LibroException.

Desde el main, se deben ingresar los datos por teclado y crear un objeto Libro, capturando posibles errores.

Ejercicio 5.

Crear una aplicación que permita calcular el promedio de una serie de notas (entre 0 y 10).

Implementar:

- Una clase `NotaInvalidaException` que se dispare cuando una nota esté fuera del rango permitido.
- Una clase `Notas` que tenga un método `calcularPromedio(List<Double> notas)`.

El método debe:

- Lanzar `NotaInvalidaException` si alguna nota no está en el rango permitido.
-
- Lanzar `ListaVacíaException` si la lista está vacía.

Ambas excepciones deben heredar de una clase base.
